UNIVERSITI TEKNIKAL MALAYSIA MELAKA
FAKULTI TEKNOLOGI DAN KEJURUTERAAN ELEKTRONIK DAN KOMPUTER

BERR 1233
DATA STRUCTURES & ALGORITHM
Sem 2 2023/2024

Group Number:15
Section Number:S2&S5
TITLE OF PROJECT : COFFEE ORDERING SYSTEM

No	Name	Matrix No	Photo
1	**THEVA PRASATH A/L MAHENDRAN**	**B122310147**	
2	**ARVIN RAU A/L APPARAO**	**B122320018**	
3	**AGILAN A/L KUMARAN**	**B122310177**	
4	**RIGNES A/P TAMIL VANAN**	**B122320069**	

FORM OF EVALUATION

COURSE : **BERR 1233**
SEM / SESSION : **2 2023/2024**
GROUP : 15
SECTION : S2&S5

No	Category	1	2	3	4	5	Marks	Total
Part A (5%)	Ideation & Knowledge background							
	Design Flow & Methodology							
	Video Demo							
Part B - *CPS (10%)	Project Requirements							
	Project Functionality							
	Quality and Creativity							
	Efficiency (Resource Utilization)							
Part C (5%)	Digital Portfolio (Personal Engagements)							
	Reflection							
	TOTAL							

COMMENTS:

Rubric of Assessment

PART	CATEGORY	MARKS				
		1	2	3	4	5
A	Ideation & Knowledge background	Not be able to utilize idea/knowledge to complete the project	Not fully utilize the idea/knowledge to complete the project	Able to utilize the idea/knowledge to complete the project	Able to utilize the idea/knowledge to create a good project	Able to fully utilize the idea/knowledge to create an attractive project
	Design Flow & Methodology	Have not done methodology and do not state the project workflow	Insufficient Methodology and do not state the project workflow	Sufficient methodology developing for the app and do not conclude in detailed the project workflow	Good methodology developing for the app and do not conclude in detailed the project workflow	Excellent methodology developing for the app and have conclude all the project workflow
	Video Demo	No video demo.	Low quality video demo. Without any voice over.	Acceptable video demo. Only demonstrate input and output.	Good video demo. Demonstrate the idea, input and output.	Excellent video demo. Demonstrate the idea, input and output creatively.
B (CPS)	Project Requirements	Integrates only 1 DS or algorithms elements. Some functions using standard built-in library.	Integrates only 2 DS or algorithms elements. Some functions using standard built-in library.	Integrates only 3 DS or algorithms elements. Some functions using standard built-in library.	Integrates only 4 DS or algorithms elements. Some functions developed using standard built-in library.	Integrates all 4 DS or algorithms elements. All functions developed in-house (not using standard built-in library).
	Project functionality	Not functioning at all.	Project functioning with major errors.	Project functioning with minor errors.	Project function perfectly. Data loaded manually.	Project function perfectly. Data pre-loaded automatically.
	Quality and Creativity	No quality and creativity	Low level quality and creativity	Acceptable quality and creativity	Good quality and creativity.	Impressive quality and high quality project.
	Efficiency (Resource utilization)	Basic programming approach (single main file only)	Insufficient approach (with a single function and main function).	Sufficient approach with only half structured and multiple functions.	Good approach with structured and multiple functions. Without modular approach.	Excellent approach with structured, multiple functions and modular approach.
C	Digital Portfolio (Personal Engagements)	Less than 5 personal engagements in indicated learning digital platforms.	Between 5-9 personal engagements in indicated learning digital platforms.	Between 10-14 personal engagements in indicated learning digital platforms.	Between 15-19 personal engagements in indicated learning digital platforms.	More than 20 personal engagements in indicated learning digital platforms.
	Reflection	No personal reflection	Basic personal reflection.	Acceptable personal reflection.	Good personal reflection.	Very good personal reflection.

Contents

INTRODUCTION 5

OBJECTIVES 6

I. Simplified Ordering Experience 6

II. Efficient Management Tools 6

III. Enhanced Customer Satisfaction 6

METHODOLOGY 7

FLOWCHART 8

SOURCE CODE 9

SAMPLE OF OUTPUT 14

Sample output if repeated order 14

Sample output if no repeated order 19

Video Link 20

INTRODUCTION

For this assignment our team presented a coffee ordering system called Coffee Shop. This Assignment specifically using C++ coding language and the application used was Visual Studio Code. This innovative system simplifies the process of managing, customizing and purchasing your perfect cup of coffee.

This Coffee Shop System simplifies the process of ordering a perfect cup of coffee with its user-friendly interface. Customers can easily navigate a wide array of options, including classic coffee types such as Americano, Espresso, Mocha, Latte, Cappuccino, and Macchiato. From choosing between hot or cold temperatures to selecting toppings like 1 shot Espresso, 2 shots Espresso, Whip cream, Caramel, Vanilla, Chocolate, Hazelnut, Almond, Cinnamon, or opting for no topping, customers can personalize their coffee to their liking. Size options range from Small, Medium, to Large, ensuring their caffeine needs are met.

Besides, the system operates efficiently for owners, managing order tracking and calculating real-time prices with transparency and accuracy. It uses a queue structure to handle up to three orders per person, maintaining order flow and preventing overload. Additionally, the system captures and sorts data on ordered coffee types, providing insights into customer preferences and popular choices.

Our system features an intuitive interface that guides you through the process of customizing your coffee, ensuring you get exactly what you crave. Start by selecting from a wide variety of coffee types, including classics like Americano, Espresso, Mocha, Latte, Cappuccino, and Macchiato. Each type of coffee is carefully crafted to cater to different tastes and preferences, ensuring there's something for everyone.

OBJECTIVES

The main objective of this project is to develop a comprehensive coffee ordering system that is efficiency for both coffee shop owners and customers. This system allows owners to manage the ordering process while providing customers to customize and place their coffee orders efficiently. The project focuses on three main objectives:

I. Simplified Ordering Experience

We've designed a system that guides customers through the coffee selection process. Whether they prefer classics like Americano or Mocha, or enjoy the richness of Espresso or Latte, customers can easily customize their coffee with options for temperature (hot or cold), toppings such as whip cream, caramel, or vanilla, and choose the ideal size that suits their mood.

II. Efficient Management Tools

Owners can track orders in real time, accurately calculate prices based on customization and manage multiple orders smoothly using a queue system. This ensures that every customer's order is handled accurately.

III. Enhanced Customer Satisfaction

Our project aim is to enhance the overall customer experience. By providing a user friendly platform that respects individual preferences and ensures orders are processed efficiently.

METHODOLOGY

To develop a Coffee Shop Ordering System using structured programming, we start by analyzing the problem. The goal is to enable customers to order different types of coffee with various customization, track these orders, calculate prices, and manage a maximum of three orders per customer. The system should display the menu, take orders, process payments, and show order history.

In the design phase, we break the system into smaller modules :

- Menu Display
- Order Taking
- Price Calculation
- Order Management
- Order History Display

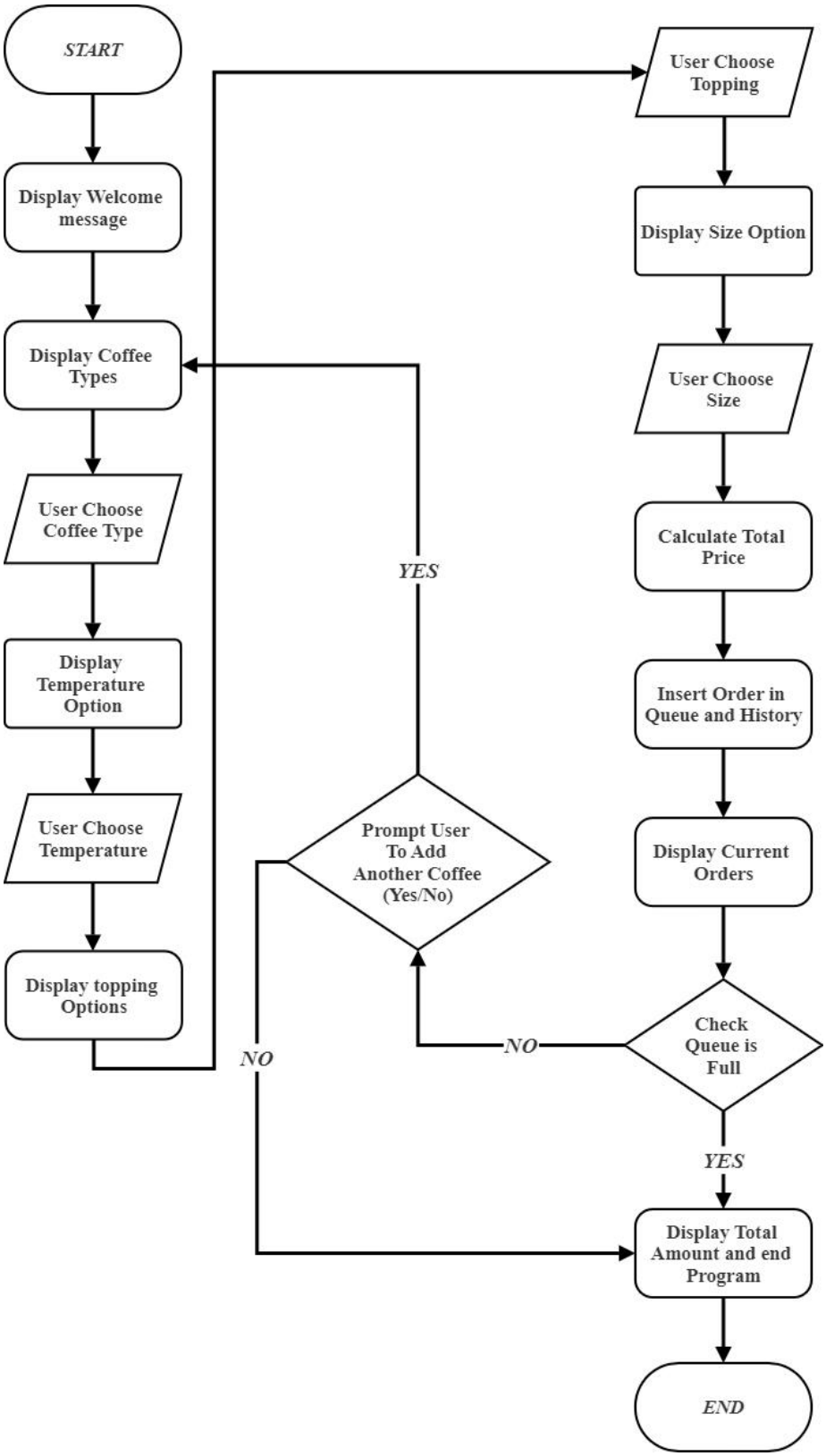
We use arrays for menu items and prices, a linked list for managing the order history and a queue for managing the order prices.

During implementation, we create functions to display the menu and options, capture user choices, insert and display orders in a linked list, and manage the queue of order prices. Error handling ensures that invalid inputs are managed, and the order limit is enforced. We then test each part of the system separately to ensure they work correctly (unit testing), check that all parts interact well together (integration testing), and validate the entire system with various scenarios (system testing).

Documentation involves adding comments in the code, creating a user manual, and documenting the system design for future maintenance. In the maintenance phase, we monitor the system, fix bugs, and update it as needed based on user feedback or new requirements.

The provided C++ code demonstrates this approach by implementing a Coffee Shop structure for menu items and prices, a linked list for order history, and a queue for order prices. The main function manages the ordering process, calculates prices, and displays the order history.

FLOWCHART


START
Display Welcome message
Display Coffee Types
User Choose Coffee Type
Display Temperature Option
User Choose Temperature
Display topping Options
User Choose Topping
Display Size Option
User Choose Size
Calculate Total Price
Insert Order in Queue and History
Display Current Orders
Check Queue is Full
NO
YES
Prompt User To Add Another Coffee (Yes/No)
YES
NO
Display Total Amount and end Program
END

SOURCE CODE

```
#include <iostream>
#include <string>
using namespace std;

const int MaxSize = 3;

struct CoffeeShop
{
    string coffeetype[6] = { "Americano", "Espresso", "Mocha", "Latte",
"Cappuccino", "Macchiato" };
    string temperature[2] = { "HOT", "COLD" };
    string topping[10] = { "1 shot Espresso", "2 shots Espresso", "Whip
cream", "Caramel", "Vanilla", "Chocolate", "Hazelnut", "Almond", "Cinnamon",
"No Topping" };
    string size[3] = { "Small", "Medium", "Large" };

    int toppingPrice[10] = { 3, 6, 1, 3, 3, 3, 3, 4, 5, 0 };
    int sizePrice[3] = { 5, 7, 9 };
    int basePrice[6] = { 6, 5, 6, 6, 7, 7 };
};

// Linked list
struct Node
{
    string coffeeType;
    Node* next;
};

struct LinkedList {
    Node* head = nullptr;

    void insert(string coffeeType)
    {
        Node* newNode = new Node();
        newNode->coffeeType = coffeeType;
        newNode->next = head;
        head = newNode;
    }

    void display()
    {
        Node* temp = head;
        while (temp)
        {
            cout << "Order: " << temp->coffeeType << endl;
            temp = temp->next;
        }
    }

    void toArray(string data[], int size)
    {
        Node* temp = head;
        int index = 0;
        while (temp && index < size)
        {
```

```
            data[index++] = temp->coffeeType;
            temp = temp->next;
        }
    }
};

void display(string data[], int size)
{
    for (int i = 0; i < size; i++)
    {
        cout << i + 1 << ". " << data[i] << endl;
    }
}

// User choice, temperature choice, topping choice, size choice
int choice(string data[], int size)
{
    int j;
    cin >> j;
    cout << "You chose: " << data[j - 1] << endl;
    return j - 1;
}

void temperatureChoice(string data[], int size)
{
    display(data, size);
    cout << "Please pick a temperature: ";
    int i;
    cin >> i;
    cout << "The temperature chosen is: " << data[i - 1] << endl;
}

int toppingChoice(string data[], int size)
{
    display(data, size);
    cout << "Please choose a topping: ";
    int i;
    cin >> i;
    cout << "The topping chosen is: " << data[i - 1] << endl;
    return i - 1;
}

int sizeChoice(string data[], int size)
{
    display(data, size);
    cout << "Please choose the size: ";
    int i;
    cin >> i;
    cout << "The size chosen is: " << data[i - 1] << endl;
    return i - 1;
}

// selection sort
void selectionSort(string data[], int size)
{
    for (int i = 0; i < size; i++)
    {
        int smallest = i;
        for (int j = i + 1; j < size; j++)
        {
```

```
            if (data[j].compare(data[smallest]) < 0)
            {
                smallest = j;
            }
        }
        string temp = data[i];
        data[i] = data[smallest];
        data[smallest] = temp;
    }
}

// Queue
struct Queue
{
    int data[MaxSize];
    int front = 0;
    int rear = -1;
    int currentSize = 0;

    void insert(int item)
    {
        if (currentSize >= MaxSize)
        {
            cout << "ORDER IS FULL! Here are the orders you already made:"
<< endl;
            display();
            return;
        }

        rear++;
        if (rear >= MaxSize)
        {
            rear = 0;
        }
        data[rear] = item;
        currentSize++;
    }

    void display()
    {
        if (currentSize <= 0)
        {
            cout << "Order is empty!" << endl;
            return;
        }

        int count = 0;
        int index = front;
        while (count < currentSize)
        {
            cout << "Order " << count + 1 << ": RM " << data[index] << endl;
            index++;
            if (index >= MaxSize)
            {
                index = 0;
            }
            count++;
        }
    }
```

```
    int totalAmount()
    {
        int total = 0;
        int index = front;
        for (int i = 0; i < currentSize; i++)
        {
            total = total + data[index];
            index++;
            if (index >= MaxSize)
            {
                index = 0;
            }
        }
        return total;
    }
};

int main()
{
    char continueOrder = 'Y';
    CoffeeShop c;
    Queue q;
    LinkedList l;

    while (continueOrder == 'Y' || continueOrder == 'y')
    {
        cout << "Welcome to our Coffee Shop\n(Per Person Only Can Order 3
Coffees)\n" << endl;

        cout << "Please choose a coffee type\n";
        display(c.coffeetype, 6);
        int coffeeTypeIndex = choice(c.coffeetype, 6);

        cout << endl;
        temperatureChoice(c.temperature, 2);

        cout << endl;
        int toppingIndex = toppingChoice(c.topping, 10);

        cout << endl;
        int sizeIndex = sizeChoice(c.size, 3);

        int totalPrice = c.basePrice[coffeeTypeIndex] +
c.toppingPrice[toppingIndex] + c.sizePrice[sizeIndex];
        cout << "The total price of your coffee is: RM " << totalPrice <<
endl;
        q.insert(totalPrice);
        l.insert(c.coffeetype[coffeeTypeIndex]);

        cout << endl;
        q.display();

        if (q.currentSize >= MaxSize)
        {
            cout << "The order is full. You cannot place more orders." <<
endl;
            cout << "The total amount for your order is: RM " <<
q.totalAmount() << endl;
```

```
            break;
        }

        cout << endl;
        cout << "Would you like to order another coffee? (Y/N): ";
        cin >> continueOrder;
    }

    if (q.currentSize > 0)
    {
        cout << "The total amount for all your coffees is: RM " <<
q.totalAmount() << endl;
    }

    cout << "Thank you for your order!" << endl;

    cout << "\nOrder History :" << endl;
    l.display();

    string coffeeTypes[5] = { "" };
    l.toArray(coffeeTypes, 5);
    selectionSort(coffeeTypes, 5);

    return 0;
}
```

SAMPLE OF OUTPUT

Sample output if repeated order

```
C:\Users\ASUS\OneDrive\Computer engineering BERR(Dgree)\Computer programming(BERN1233)\C++ Programming\output\x64\Debug\output.exe
Welcome to our Coffee Shop
(Per Person Only Can Order 3 Coffees)

Please choose a coffee type
1. Americano
2. Espresso
3. Mocha
4. Latte
5. Cappuccino
6. Macchiato
```

The program begin with greetings for user to the coffee shop.The program displays to the user to choose a coffee type for order and it also states the condition which is per person can only order 3 coffees at a time.

```
C:\Users\ASUS\OneDrive\Computer engineering BERR(Dgree)\Computer programming(BERN1233)\C++ Programming\output\x64\Debug\output.exe
Welcome to our Coffee Shop
(Per Person Only Can Order 3 Coffees)

Please choose a coffee type
1. Americano
2. Espresso
3. Mocha
4. Latte
5. Cappuccino
6. Macchiato
1
You chose: Americano

1. HOT
2. COLD
Please pick a temperature:
```

Upon from selection by the user,the program then shows the temperature of coffee to pick by user.

```
Please pick a temperature: 1
The temperature chosen is: HOT

1. 1 shot Espresso
2. 2 shots Espresso
3. Whip cream
4. Caramel
5. Vanilla
6. Chocolate
7. Hazelnut
8. Almond
9. Cinnamon
10. No Topping
Please choose a topping:
```

After choosing the temperature ,the program shows the available toppings that it has to offer the user and it also requires the user to choose the topping.

```
Please choose a topping: 5
The topping chosen is: Vanilla

1. Small
2. Medium
3. Large
Please choose the size:
```

After picking the topping by user, the program displays size of coffee which is medium, small or large. And then user needs to choose.

```
Please choose the size: 1
The size chosen is: Small
The total price of your coffee is: RM 14

Order 1: RM 14

Would you like to order another coffee? (Y/N):
```

Then, it displays the choice of user and the total amount of coffee and it also displays to order another coffee until maximum 3 coffee per person

```
Would you like to order another coffee? (Y/N): y
Welcome to our Coffee Shop
(Per Person Only Can Order 3 Coffees)

Please choose a coffee type
1. Americano
2. Espresso
3. Mocha
4. Latte
5. Cappuccino
6. Macchiato
```

If the user enters yes, the process for ordering will be repeated from choose a coffee type.

```
1
You chose: Americano

1. HOT
2. COLD
Please pick a temperature: 1
The temperature chosen is: HOT

1. 1 shot Espresso
2. 2 shots Espresso
3. Whip cream
4. Caramel
5. Vanilla
6. Chocolate
7. Hazelnut
8. Almond
9. Cinnamon
10. No Topping
Please choose a topping:
```

Then, the process for temperature and topping also will be repeated

```
Please choose a topping: 3
The topping chosen is: Whip cream

1. Small
2. Medium
3. Large
Please choose the size: 2
The size chosen is: Medium
The total price of your coffee is: RM 14

Order 1: RM 14
Order 2: RM 14

Would you like to order another coffee? (Y/N):
```

Since the max size is 3, the program will display the yes or no option to make sure the user needs another coffee or not.

```
Would you like to order another coffee? (Y/N): y
Welcome to our Coffee Shop
(Per Person Only Can Order 3 Coffees)

Please choose a coffee type
1. Americano
2. Espresso
3. Mocha
4. Latte
5. Cappuccino
6. Macchiato
5
You chose: Cappuccino

1. HOT
2. COLD
Please pick a temperature: 2
The temperature chosen is: COLD
```

If the user enters yes, the process will be repeated again from the choose a coffee type display.

```
The temperature chosen is: COLD

1. 1 shot Espresso
2. 2 shots Espresso
3. Whip cream
4. Caramel
5. Vanilla
6. Chocolate
7. Hazelnut
8. Almond
9. Cinnamon
10. No Topping
Please choose a topping: 10
The topping chosen is: No Topping

1. Small
2. Medium
3. Large
Please choose the size: 1
The size chosen is: Small
The total price of your coffee is: RM 12

Order 1: RM 14
Order 2: RM 14
Order 3: RM 12
The order is full. You cannot place more orders.
The total amount for your order is: RM 40
The total amount for all your coffees is: RM 40
Thank you for your order!
```

Then, the process for temperature and topping also will be repeated again and because of the max order per person is 3 ,the program shows that the order is full and cannot place more orders and display the coffee amount and the total amount of all coffees

```
Order History :
Order: Cappuccino
Order: Americano
Order: Americano
```

Lastly the program end with thank you for your order and display the order history.

Sample output if no repeated order

```
C:\Users\ASUS\OneDrive\Computer engineering BERR(Dgree)\Computer programming(BERN1233)\C++ Programming\output\x64\Debug\output.exe
Welcome to our Coffee Shop
(Per Person Only Can Order 3 Coffees)

Please choose a coffee type
1. Americano
2. Espresso
3. Mocha
4. Latte
5. Cappuccino
6. Macchiato
1
You chose: Americano

1. HOT
2. COLD
Please pick a temperature: 2
The temperature chosen is: COLD
```

The program begin with greetings for user to the coffee shop.The program displays to the user to choose a coffee type for order and it also states the condition which is per person can only order 3 coffees at a time. on from selection by the user, the program then shows the temperature of coffee to pick by user

```
The temperature chosen is: COLD

1. 1 shot Espresso
2. 2 shots Espresso
3. Whip cream
4. Caramel
5. Vanilla
6. Chocolate
7. Hazelnut
8. Almond
9. Cinnamon
10. No Topping
Please choose a topping: 5
The topping chosen is: Vanilla

1. Small
2. Medium
3. Large
Please choose the size: 1
The size chosen is: Small
```

Upon from selection by the user,the program then shows the temperature of coffee to pick by user after choosing the temperature, the program shows the available toppings that it has to offer the user and it also requires the user to choose the topping. Then, the program displays size of coffee which is medium, small or large and then user needs to choose.

```
The size chosen is: Small
The total price of your coffee is: RM 14

Order 1: RM 14

Would you like to order another coffee? (Y/N): n
```

After choosing the size the total price of coffee will appeared and it also displays whether user needs to order another coffee or not.

```
Would you like to order another coffee? (Y/N): n
The total amount for all your coffees is: RM 14
Thank you for your order!

Order History :
Order: Americano
```

If no, then it automatically display the total amount of all coffee and ends with thank you for your order and it displays the order history.

Video presentation link : https://youtu.be/PPYvkgPHnC8?si=N9YuCSyx_AlLL4Ac

Thank You